Arrays →

Advantages →

- ↳ store multiple element of same type with one single variable name
- ↳ Accessing elements is fast as long as you have an index, as opposed linked list where you have to traverse from the head.

Disadvantages

- ↳ Addition and removal of element into/from the middle of an array is slow because the remaining element need to be shifted to accomodate the new/missing element. An exception of this is at the end of the array.
- ↳ For certain languages where array sizes is fixed, it cannot alter its size after initialization. if an insertion causes the total number of element to exceed the size, a new array has to be allocated and the existing elements have to copied over. The act of creating a new array and tranferring elements over takes O(n) time.

Common terms

- ↳ Subarray - A range of contiguous value within an array
  - Example an array of contiguous values within an array [2,3,6,1,5,4], [3,6,1] is a subarray while [3,1,5] is not a subarray.
- ↳ Subsequence - A sequence that can be derived from the given sequence by deleting some or no element without changing the order of the remaining elements.
  - Example: given an array [2,3,6,1,5,4], [3,1,5] is a subseq - merge but [3,5,1] is not a subsequence.

Operations	Big-O	Notes
Access	O(1)	
Search	O(n)	
Search(Sorted)	O(log(n))	
Insert	O(n)	Insertion would require shifting all the subsequent element to the right by one and that takes O(n)
Insert(at the end)	O(1)	Special case of insertion where no other element need to shifted.

Operations	Big-O	Notes
Remove	O(n)	Removal would require shifting all the subsequent elements to the left by one & that takes O(n).
Remove (at the end	O(1)	Special case of remove where no other element need to be shifted.

Thing to look out for during interviews

- clarify if there are duplicates value in the array. Would the presence of duplicate value affect the answer? Does it make the question simpler or harder?

  `if( i > 0 && arr[i] == arr[i-1]) continue;` → skips Dups

- When using an index to iterate through array element, be careful not to go out bound.

general rule for calculating Range →

Your Goal	Start from	Why?
you need to compare arr[i] with arr[i+1]	$i = n-2$	Because $i+1$ must be $\leq n-1$
you need to compare arr[i] with arr[i-1]	$i = 1$	because $i-1$ must be ≥ 0
Normal Scan	$i = 0$	first element is included
reverse normal Scan	$i = n-1$	start at last element

Comman Access Pattern

Access Pattern ⇝ $[0 \leq idx < arr.size()]$

		Safe start/End
arr[i] ⇝	Required Range for i $[0 \leq i < n]$	⇝ any
arr[i+1] ⇝	$0 \leq i < n-1$	⇝ start at $n-2$ if going backwar
arr[i-1] ⇝	$1 \leq i < n$	⇝ start at 1 if going forward
arr[i+K] ⇝	$0 \leq i < n-K-1$	⇝ $i \leq n-K-1$
arr[i-K] ⇝	$K \leq i < n$	⇝ $i \geq K$
arr[left], arr[right] ⇝	$0 \leq left \leq right < n$	⇝ left = 0, right = n-1
arr[mid-1], arr[mid+1] ⇝	$1 \leq mid \leq n-2$	mid = (low + high)/2 where low ≥ 1 & high ≤ n-2

- Be mindful about slicing or concatenating arrays in your code. Typically slicing and concatenation array would take O(n) time. Use start and end indices to demarcat a subarray/range where possible

Corner Cases

- Empty Sequence
- Sequence with 1 or 2 element
- Sequence with repeated Element
- Duplicates values in the sequence.

Techniques

↳ Note that because both array and strings are sequence (a string is an array of characters), most of techniques here will apply to string problems.

1) Sliding Window

↳ This applies to many subarray/substring problems.

In a sliding window, the two pointers usually move in the same direction will never overtake each other This ensure that each value is only visited at most twice and the time Complexity is still O(n).

2) Two pointers

↳ This pointer is a more general version of sliding Window where the pointer can cross each other and can be on different arrays.

When you are given two array to process, its comman to have one index per array (pointer) to traverse/compare the both of them, incrementing one of the pointers when relevant.

3) Traversing from the right.

Sometimes you can traverse the array starting from the right instead of the conventional approach of the left.

4) Sorting the Array

Is the array sorted or partially sorted? if it is, some form of binary search be possible. This also usually mean that the interviewer is looking for a solution that is faster than O(n).

can you sort the array? Sometimes sorting the array first may significantly simplify the problem. Obviously this would not work if the order of array element need to be preserved.

5) Precomputation

For questions where summation or multiplication of a subarray is involved, pre-computation using hashing or a prefix/suffix sum/product might be useful.

6) Index as a hash key

if you are given a sequence and the interviewer asks for a O(1) space, it might be possible to use the array itself as a hash table. for example → if the array only has value from 1 to N, where N is the length of the array, negate the value at that index (minus one) to indicate presence of presence of that number.

7) Traversing the array more than Once

This might be obvious, but traversing the array twice/thrice (as long as fewer than n times) is still O(n). Sometimes traversing the array more than once can help you solve the problem while keeping the time complexity to O(n).